# MZ2POL-99: Best Practice Summary

> **Series:** MZ2POL | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 03/26/2026

## Overview

This notebook consolidates every actionable best practice from the MZ2POL series (notebooks 00-08) into a single definitive reference. Each practice includes the exact setting or value to use, a priority level, and its category. Use this as a pre-flight checklist before, during, and after your migration from Management Zones to Policies, Boundaries, and Segments.

---

## Table of Contents



---

## 1. Assessment and Planning

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Classify every MZ by type before migration | Categories: Team/Ownership, Environment, Geographic, Application, Infrastructure | Critical |
| Prioritize migration by impact | P1: Security/compliance MZs, P2: Team-based MZs, P3: Filtering-only MZs, P4: Informational/legacy MZs | Critical |
| Audit entity security context coverage before starting | Run `countIf(isNotNull(dt.security_context))` on services, hosts, and process groups; target 100% | Critical |
| Map each MZ to its replacement construct | Team MZ -> Security Context + Policy + Boundary; Environment MZ -> Boundary; Region MZ -> Segment; Application MZ -> Segment | Critical |

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Document all MZ dependencies before migration | Inventory alerting profiles, dashboards, API integrations, SLOs, and automation workflows referencing each MZ | Critical |
| Use MZ2POL-00 SDK tool for MZ inventory | Run the TypeScript analysis tool against Settings API `builtin:management-zones` to export all rules in flat table format | Recommended |
| Identify tag patterns for segment creation | Query `fetch dt.entity.host \| expand tag = tags \| parse tag, "LD:tagKey ':' LD:tagValue"` to discover common tag keys | Recommended |
| Perform risk assessment per MZ | Score each MZ on likelihood and impact for: users lose access, wrong data visible, alert routing breaks, dashboard filters fail | Recommended |

## 2. Security Context

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Set `dt.security_context` on every entity before creating boundaries | Assign via Entity Enrichment rules (preferred), OneAgent host properties, or OpenPipeline processing | Critical |
| Use a consistent naming convention for security context values | `team-{name}` for teams, `env-{name}` for environments, `region-{code}` for regions, `app-{name}` for applications | Critical |
| Derive security context from existing tags | Map `tag team:frontend` to `dt.security_context = "team-frontend"` to avoid double-entry | Recommended |
| Achieve 100% security context coverage before cutover | Any entity without `dt.security_context` becomes invisible to boundary-scoped users | Critical |
| Use combined naming for multi-dimensional scoping | Format: `{env}-{team}` (e.g., `prod-frontend`) when access requires both environment and team dimensions | Recommended |
| Automate security context for new entities | Configure auto-tagging rules or Entity Enrichment so newly deployed services/hosts get security context automatically | Critical |
| Leverage primary Grail fields for context | Use `k8s.cluster.name`, `k8s.namespace.name`, `aws.account.id`, `dt.host_group.id` as security context sources for Kubernetes/cloud workloads | Recommended |

## 3. Policy Design

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Start with default policies; customize only when needed | `Dynatrace Viewer` for read-only, `Dynatrace Standard User` for regular users, `Dynatrace Professional User` for power users, `Dynatrace Admin User` for admins | Critical |
| Pair a Dynatrace access policy with a data access policy | Viewers -> `Dynatrace Viewer + Data Viewer`; Developers -> `Dynatrace Standard + Data Viewer`; SRE -> `Dynatrace Professional + Data Editor`; Admins -> `Dynatrace Admin + Data Editor` | Critical |

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Apply least privilege | Grant the minimum permissions required for each role; never use `ALLOW storage:*:*` without a boundary or condition | Critical |
| Use policy conditions with `WHERE` for scoped access | `ALLOW storage:logs:read WHERE storage:dt.security_context = "team-frontend"` | Recommended |
| Never create one policy per user | Assign policies to groups; bind users to groups | Critical |
| Never duplicate default policies | Extend with custom policies only for permissions default policies do not cover | Recommended |
| Use descriptive policy names | Format: `"Frontend Team - Standard Access"` not `"Policy 1"` | Recommended |
| Document every custom policy's purpose | Include description field when creating the policy via UI or API | Recommended |

## 4. Boundary Design

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Include all three domains in every boundary | `environment:management-zone IN ("LOB5");` + `storage:dt.security_context IN ("LOB5");` + `settings:dt.security_context IN ("LOB5");` | Critical |
| Keep boundary conditions simple | One condition per line; each line is OR-combined within the boundary | Critical |
| Stay within the 10-line limit per boundary | Create multiple boundaries if you need more than 10 conditions | Critical |
| Use `startsWith` for environment/region prefixes | `storage:dt.security_context startsWith "prod-"` to match all production contexts | Recommended |
| Use `IN` for multi-value restrictions | `storage:dt.security_context IN ("team-infra", "team-sre", "team-platform")` for platform teams boundary | Recommended |
| Create reusable boundaries | One boundary should serve multiple policy bindings; avoid one-off boundaries | Recommended |
| Use descriptive boundary names | Format: `"Production Environment"`, `"LOB5-Scope"`, `"Frontend Team Scope"` | Recommended |
| Never omit the `settings:dt.security_context` domain | Without it, settings objects remain unrestricted even when storage is scoped | Critical |
| Never omit the `storage:dt.security_context` domain | Without it, Grail data (logs, spans, metrics) remains unrestricted even when classic entities are scoped | Critical |

## 5. Segment Design

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Use segments for data filtering; use policies+boundaries for access control | Segments replace MZ filtering; policies+boundaries replace MZ permissions. Never conflate the two. | Critical |
| Align segments with business structure | Create segments per team, product, region, or environment — not per technical component | Critical |
| Use `matchesValue(tags, "key:value")` for tag-based segment filters | Replaces MZ rule `host tag equals value` | Critical |
| Use `matchesPhrase(dt.entity.service.name, "text")` for service name filters | Replaces MZ rule `service name contains` | Recommended |
| Use variables for dynamic segments | Entity variable: `filter dt.entity.kubernetes_cluster == $cluster`; List variable: `filter matchesValue(tags, concat("env:", $environment))` | Recommended |
| Test segment filter logic with DQL before creating the segment | Run the filter as a standalone DQL query and verify results match expected entity set | Critical |
| Use consistent segment naming | `"Production Environment"`, `"Frontend Team Services"`, `"North America Region"` — not `"Segment 1"` | Recommended |
| Share segments appropriately | Private for personal use, shared for team use, public for org-wide use | Recommended |
| Use indexed fields in segment filters | Tags and entity IDs are indexed; avoid regex on large datasets | Recommended |
| Layer multiple segments for precise filtering | Select multiple segments in the app header for multi-dimensional filtering (e.g., team + environment) | Optional |

## 6. Bucket Strategy

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Use buckets for physical data isolation (compliance, strict team separation) | Create team-specific or compliance buckets: `pci_logs`, `frontend_logs`, `prod_spans` | Critical |
| Use segments (not buckets) for ad-hoc or regional filtering | Buckets are irreversible; segments are flexible. Only use buckets when physical isolation is required. | Critical |
| Plan bucket names carefully — they are immutable | Naming convention: `{team}_{datatype}` or `{env}_{datatype}`. 3-100 chars, lowercase alphanumeric, underscores, hyphens only. | Critical |
| One data type per bucket | Logs, metrics, events, or spans — never mixed in a single bucket | Critical |

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Stay within 80 buckets per environment | Default platform limit | Critical |
| Target ~1 TB/day per bucket for optimal query performance | Acceptable: 1-3 TB/day (limited query window). Hard limit: 3 TB/day per bucket. | Recommended |
| Create buckets and configure OpenPipeline routing BEFORE migrating policies | Data must flow to correct buckets first; policies reference bucket names | Critical |
| Reference buckets in both policies and boundaries | Policy: `ALLOW storage:logs:read WHERE storage:bucket.name = "frontend_logs"`. Boundary: `storage:bucket.name IN ("frontend_logs", "frontend_spans")` | Recommended |
| Do not use buckets for regional or application MZs | Use segments instead — buckets cannot be consolidated or split later | Recommended |

## 7. Group and SAML Structure

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Use SAML groups tied to Active Directory for team management | Map AD groups to Dynatrace groups via SAML assertion; no separate user management needed | Critical |
| Structure: Group -> Policy + Boundary | `Group: "LOB5-Team" (SAML) -> Policy: Dynatrace Professional + Boundary: LOB5-Scope (three-domain)` | Critical |
| For stricter isolation, apply boundary to all policies on a group | Both `Dynatrace Viewer` and `Dynatrace Professional` get the same boundary | Recommended |
| Create one group per MZ-equivalent scope | `frontend-team`, `prod-viewers`, `sre-team` — one group per access boundary | Critical |
| Never assign permissions to individual users | Always use group-based policy bindings | Critical |
| Document group-to-MZ mapping during migration | Maintain a table: MZ name -> new group name -> policy -> boundary | Recommended |

## 8. Templated Policies at Scale

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Use `${bindParam:...}` templated policies instead of one policy per MZ | 3-5 templates replace 92+ individual policies. Template: `ALLOW storage:*:read WHERE storage:dt.security_context = "${bindParam:team}"` | Critical |
| Create templates by MZ type, not by MZ instance | `tpl-team-data-reader`, `tpl-team-data-editor`, `tpl-region-reader` — one template per access pattern | Critical |

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Use Pattern A (read-only) for viewer MZ replacements | Template grants `storage:logs:read`, `storage:spans:read`, `storage:metrics:read` scoped by `${bindParam:team}` | Recommended |
| Use Pattern B (full access) for editor MZ replacements | Template grants `storage:*:read`, `storage:*:write`, `settings:objects:*` scoped by `${bindParam:team}` | Recommended |
| Use multiple bindings of the same template for multi-MZ users | Bind template twice to same group with different parameter values (e.g., `team: checkout` and `team: shared-services`); bindings are additive | Recommended |
| Use `startsWith` in templates for region/environment patterns | `WHERE storage:dt.security_context startsWith "${bindParam:region-prefix}"` | Optional |
| Automate bulk binding via IAM API | `POST /iam/v1/repo/account/{ACCOUNT}/bindings/{POLICY_UUID}/{GROUP_UUID}` with `{"parameters": {"team": "value"}}` | Recommended |
| Verify all bindings after bulk creation | `GET /iam/v1/repo/account/{ACCOUNT}/bindings/{POLICY_UUID}` and validate HTTP 201/409 responses | Critical |

## 9. Migration Execution

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Follow the 7-phase migration order | Phase 1: Foundation -> Phase 2: Security Context -> Phase 3: Policies/Boundaries -> Phase 4: Segments -> Phase 5: Parallel Running -> Phase 6: Cutover -> Phase 7: Cleanup | Critical |
| Run parallel (MZ + new model) for 2-4 weeks before cutover | Both systems active simultaneously; users should see identical data via both paths | Critical |
| Compare entity counts between MZ and segment during parallel running | `countIf(in(managementZones, {"MZ-Name"}))` vs `countIf(matchesValue(tags, "key:value"))` — counts must match | Critical |
| Update dashboards to use segments before cutover | Replace MZ filters with segment selectors at dashboard level and tile level | Critical |
| Update alerting profiles to use segments before removing MZs | Alerts scoped to MZs will break when MZs are removed | Critical |
| Update API integrations to use new access model before cutover | API calls with MZ parameters must be converted to policy-based authentication | Recommended |
| Archive MZ configurations before deletion | Export via Settings API and store in version control | Recommended |

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Wait 2+ weeks after cutover before deleting MZs | Verify no remaining MZ dependencies in alerting, dashboards, or API integrations | Critical |
| Send cutover communication to all users | Include: what changed, action required (select segments in apps), documentation link, support contact | Recommended |
| Prepare a rollback plan | Document how to re-enable RBAC MZ assignments if critical access issues arise post-cutover | Critical |

## 10. Validation and Ongoing Maintenance

| Practice | Recommended Setting/Value | Priority |
|---|---|---|
| Validate security context coverage reaches 100% | `fetch dt.entity.service \| summarize total = count(), covered = countIf(isNotNull(dt.security_context))` | Critical |
| Compare MZ membership vs security context alignment per MZ | Query entities `in(managementZones, {"MZ"}) AND dt.security_context == "expected-value"` — all must match | Critical |
| Find entities in MZ but missing from segment | `filter in(managementZones, {"MZ"}) \| filter NOT matchesValue(tags, "key:value")` — result must be empty | Critical |
| Test each user type with a test account | Log in as test user, verify expected data visible, verify restricted data hidden | Critical |
| Audit security context weekly | Run coverage query to catch new entities deployed without security context | Recommended |
| Review segment effectiveness monthly | Verify segments return expected entity counts and data | Recommended |
| Perform access review quarterly | Verify user group memberships, policy bindings, and boundary conditions are still appropriate | Recommended |
| Monitor for untagged entities | `fetch dt.entity.service \| filter isNull(dt.security_context) \| filter NOT matchesValue(tags, "team:*")` — track count on a dashboard | Recommended |
| Build an access health dashboard | Include KPIs: security context coverage %, entities per context, untagged entity count | Optional |
| Automate onboarding for new entities | Auto-tagging + Entity Enrichment rules ensure new services/hosts inherit correct security context | Recommended |

---